



Learning to Classify: A Flow-Based Relation Network for Encrypted Traffic Classification

Wenbo Zheng

School of Software Engineering
Xi'an Jiaotong University
Xi'an, China

Lan Yan

Institute of Automation
Chinese Academy of Sciences
Beijing, China

Chao Gou*

School of Intelligent Systems Engineering
Sun Yat-sen University
Guangzhou, China
gouchao@mail.sysu.edu.cn

Shaocong Mo

College of Computer Science and Technology
Zhejiang University
Hangzhou, China

ABSTRACT

As the size and source of network traffic increase, so does the challenge of monitoring and analyzing network traffic. The challenging problems of classifying encrypted traffic are the imbalanced property of network data, the generalization on an unseen dataset, and overly dependent on data size. In this paper, we propose an application of a meta-learning approach to address these problems in encrypted traffic classification, named Flow-Based Relation Network (RBRN). The RBRN is an end-to-end classification model that learns representative features from the raw flows and then classifies them in a unified framework. Moreover, we design "hallucinator" to produce additional training samples for the imbalanced classification, and then focus on meta-learning to classify unseen categories from few labeled samples. We validate the effectiveness of the RBRN on the real-world network traffic dataset, and the experimental results demonstrate that the RBRN can achieve an excellent classification performance and outperform the state-of-the-art methods on encrypted traffic classification. What is more interesting, our model trained on the real-world dataset can generalize very well to unseen datasets, outperforming multiple state-of-art methods.

CCS CONCEPTS

• **Networks** → **Network management**; • **Computing methodologies** → **Artificial intelligence**; • **Information systems** → *Data management systems*; World Wide Web.

KEYWORDS

traffic classification, meta-learning, relational networks, data augmentation

*Corresponding Author

This paper is published under the Creative Commons Attribution 4.0 International (CC-BY 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.

WWW '20, April 20–24, 2020, Taipei, Taiwan

© 2020 IW3C2 (International World Wide Web Conference Committee), published under Creative Commons CC-BY 4.0 License.

ACM ISBN 978-1-4503-7023-3/20/04.

<https://doi.org/10.1145/3366423.3380090>

ACM Reference Format:

Wenbo Zheng, Chao Gou, Lan Yan, and Shaocong Mo. 2020. Learning to Classify: A Flow-Based Relation Network for Encrypted Traffic Classification. In *Proceedings of The Web Conference 2020 (WWW '20)*, April 20–24, 2020, Taipei, Taiwan. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3366423.3380090>

1 INTRODUCTION

Recently, traffic encryption has been widely used on the Internet due to advanced encryption technology. Encryption technology not only protects the freedom, privacy, and anonymity of Internet users but also makes the users evade the detection by the firewall and circumvent the surveillance systems. However, encryption technology is also used to obtain the unscrupulous profits of the opponent. For example, an attacker encrypts malware traffic to invade and attack the system anonymously. Besides, criminals use privacy-enhancing tools (for example, Tor [18]) to penetrate the dark network, where they can buy drugs, weapons, and forged documents (such as passports, driver's licenses, and media that provide contract killers to attract customers [26]). That is to say, the abuse of encryption technology brings new threats to network security and network management [11]. Therefore, the identification and classification of encrypted traffic have aroused great concern in academia and industry [25].

Traffic classification techniques have evolved significantly over time. The first and easiest approach uses port numbers. However, its accuracy has declined because newer applications either use well-known port numbers to disguise their traffic or avoid using standard registered port numbers. Despite its inaccuracy, the port number is still widely used either alone or in tandem with other features in practice [13]. The next generation of traffic classifiers, relying on the payload, called data packet inspection (DPI), focuses on finding patterns or keywords in data packets. These methods are only applicable to unencrypted traffic and have high computational overhead. As a result, a new generation of methods, based on flow statistics, emerged [10]. These methods rely on statistical or time series features, which enable them to handle both encrypted and unencrypted traffic. These methods usually employ classical machine learning (ML) algorithms, such as random forest (RF) and k-nearest neighbor (KNN) [14]. However, their performance heavily depends

on human-engineered features, which limit their generalizability [20].

Deep learning obviates the need to select features by a domain expert because it automatically selects features through training. This characteristic makes deep learning a highly desirable approach for traffic classification, especially when new classes always emerge and patterns of old classes evolve [2, 21]. Another essential characteristic of deep learning is that it has a considerably higher capacity of learning in comparison to traditional ML methods [4], and thus can learn highly complicated patterns. Combining these two characteristics, as an end-to-end approach, deep learning is capable of learning the nonlinear relationship between the raw input and corresponding output without the need to break the problem into small sub-problems of feature selection and classification [16, 34].

However, there are three main challenges in traffic classification, using deep-learning-based approaches:

- (1) Deep-learning-based approaches cannot solve the problem of class unbalanced [12, 29];
- (2) Deep-learning-based approaches cannot adapt well to the new environment, and cannot be used to generalize on unseen dataset [5, 23];
- (3) Performances of deep-learning-based approaches are overly dependent on data size [32, 36].

Given a single data of a novel concept (category), a person can visualize what this data would look like in other different surroundings. If computer recognition systems could do such hallucination, they might be able to learn novel concepts (categories) from less data. As humans, our knowledge of these shared modes of variation may allow us to visualize what a novel object might look like in other surroundings. If machine systems could do such “hallucination” or “imagination”, then the hallucinated examples could be used as additional training data to build better classifiers. *Why not use the “hallucinator” to produce additional training samples for the classification?*

The ability to rapidly learn and generalize from a small number of examples is a critical characteristic of human intelligence because humans can leverage the prior knowledge obtained from the previous learning experience. Although current deep learning approaches have achieved significant success in many tasks, massive labeled data and excessive training time are still required, because each task is independently considered and the model parameters are learned from scratch without incorporating task-specific prior knowledge. How to extract prior knowledge and transfer them to unseen tasks with limited data has become active research areas in machine learning. Recently, meta-learning has emerged as a kind of promising approach to solve this problem. A generic meta-learning framework usually contains a meta-level learner and a base-level learner. The base-level learner is designed for specific tasks, such as classification, regression, and neural network policy. The meta-level learner aims to learn prior knowledge across different tasks. The prior knowledge can be transferred to the base-level learner to help quickly adapt to similar unseen tasks. *Therefore, in this paper, we focus on meta-learning, which aims at learning to classify unseen categories from few labeled samples. By the strategy of meta-learning or few-shot learning, we are able to the second and third above challenges.*

In this paper, we propose an end-to-end model named a **Flow-Based Relation Network (RBRN)** for encrypted traffic classification. The RBRN learns representative features from the raw flow sequences rather than manually designed features. The major structure of the RBRN model includes a hallucinator to produce additional training samples, an encoder to generate the features, a decoder to restore the input sequences and a meta-learning-based classifier to recognize applications. Hallucinator maps real flow sequences to hallucinated flow sequences. Both the encoder and decoder are the multi-layer convolutional neural networks to handle the input sequences with different flow lengths. The features for classification are learned automatically from the raw flow sequences by the encoder and the decoder. Besides, the decoder also learns features to enhance the discrimination of flows. RBRN build the two-branch relation network via meta-learning to address the classification. The hallucinator, encoder, decoder, and meta-classifier are jointly trained with the raw flow sequences and application labels. The flexible architecture increases the generalization of our RBRN model.

Our contributions can be briefly summarized as follows:

- (1) We propose an end-to-end RBRN model for the encrypted traffic classification. The RBRN jointly learns features from the raw flow sequences and makes classifier to identify flows, consists of a hallucinator, an encoder, a decoder, and a meta-classifier.
- (2) Incorporating this ability to hallucinate novel instances (flow sequences) of new concepts (categories) is able to improve the performance of our model, i.e., learning concepts (categories) from few examples to classify.
- (3) Our RBRN achieves excellent results on the real-world network traffic data for the encrypted traffic classification and outperforms several state-of-the-art methods.
- (4) We empirically show that the proposed RBRN has strong robustness and outperforms existing methods in encrypted traffic classification on unseen datasets.

2 PRELIMINARIES

In this section, we first give the definition of the encrypted traffic classification problem. Then, the generative flow model and the autoencoder framework are introduced briefly.

2.1 Problem Definition

We consider the problem of encrypted traffic classification as meta-classifier learning. There are three datasets: a training set, a support set, and a testing set. Note that the support set and testing set share the same label space, but the training set has its own label space. In other words, the training set is disjoint with the support set and testing set. We focus on classifying the encrypted traffic into specific applications with the flow sequences as the only raw traffic information. A raw flow can be represented as several sequences with the same flow length and different types (e.g., message type sequences and packet length sequences). In general, we consider one kind of sequences as the flow sequences, and other sequences can be used in the same way. Assume that there are M samples and N applications in total.

In principle, we can train a classifier to assign a class label $\hat{y}$ to each sample $\hat{x}$ in the test set while we only use the support

set. However, in most cases, the performance of such a classifier is usually not excellent because of the lack of the labeled samples in the support set. Therefore, we use the meta-learning on the training set to transfer the extracted knowledge to the support set. It aims to perform the few-shot learning on the support set better and classify the test set more successfully.

We propose a novel matching network [19, 22] to solve the problem of encrypted traffic classification. Suppose that there are m labeled samples for each of n unique classes in support set. We select randomly n classes from the training set with m labeled samples from each of the n classes to conduct the sample set $S = \{(x_i, y_i)\}_{i=1}^z$ ($z = m \times n$; $m \in [0, M]$; $n \in [0, N]$), and we select the remaining samples to conduct the query set $Q = \{(x_j, y_j)\}_{j=1}^v$. This split strategy of sample and query set aims to simulate the support and test set that will be encountered at test time.

2.2 Glow Model

Glow model [7] contains three parts: ActNorm, 1×1 invertible convolution and affine coupling layer, as shown in Figure 1. Here, I signifies the input of the layer, and o signifies its output. Both I and o are tensors of shape $[h \times w \times c]$ with spatial dimensions (h ; w) and channel dimension c . With (i, j) we denote spatial indices into tensors I and o . The function $\mathcal{F}$ is a nonlinear mapping, such as a (shallow) convolutional neural network. In total, the process of Glow model follows as:

```

1: function ACTNORM
2:    $\forall i, j : o_{i,j} = s \odot I_{i,j} + b$ 
3: end function
4: function  $1 \times 1$  INVERTIBLE CONVOLUTION
5:    $\forall i, j : o_{i,j} = W I_{i,j}$ 
6: end function
7: function AFFINE COUPLING LAYER
8:    $I_a, I_b = \text{split}(I)$ 
9:    $(\log s, t) = \mathcal{F}(I_b)$ 
10:   $s = \exp(\log s)$ 
11:   $o_a = s \odot I_a + t$ 
12:   $o_b = I_b$ 
13:   $o = \text{concat}(I_a, I_b)$ 
14: end function

```

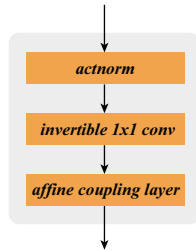


Figure 1: One step of Glow model, which consists of an ActNorm step, followed by an invertible 1×1 convolution, then followed by an affine coupling layer[7].

ActNorm The first layer in our model is the ActNorm layer, which is called Activation Normalization, and the overall effect is similar to batch normalization. It is a fact that the variance of

the active noise added by batch normalization is inversely proportional to the small batch size of the GPU or other processing units (PU), which causes performance degradation. Therefore, we use the ActNorm layer. Besides, ActNorm performs an active affine transformation using the scale and deviation parameters for each channel, similar to batch normalization. We initialize these parameters, given the small batch size of the initial data, so that the behavior after each channel has zero mean and unit variance. After initialization, the scale and deviation are treated as regular trainable parameters that are independent of the data. In other words, ActNorm is the data preprocess of the input data.

Affine Coupling Layer The mapping of the affine coupling layer is the key to realize the reversible transformation. The idea of affine coupling is divided into three parts:

Zero Initialization: Initialize the last convolution of each $\mathcal{F}$ with zeros so that each affine coupling layer initially performs an identity function, which helps train the deep network.

Split and Concatenation: The *split()* function divides the input tensor into two halves along the channel dimension, and the *concat()* operation performs the corresponding reverse operation: concatenating into a single tensor. In Glow model, splitting is performed only along the channel dimension, and then by this way, we can simplify the overall architecture.

Permutation: Each of the above process steps should be sorted with variables to ensure that each dimension can affect every other dimension after sufficient process steps.

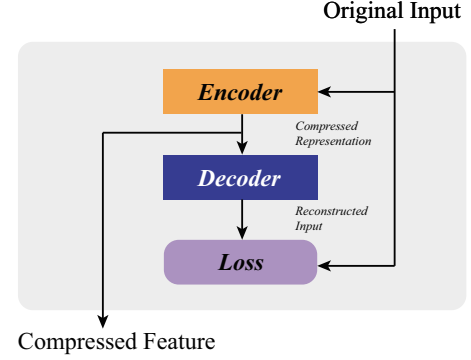


Figure 2: The total process of autoencoder. Encoder compresses original input into code as compressed representation. The following decoder restores the code to the input as reconstructed data. In addition, encoder and decoder generate compressed features.

1×1 Invertible Convolution The 1×1 invertible convolution is an improvement on NICE [17] and RealNVP [6] that reverses the ordering of the channels, while the 1×1 invertible convolution replaces the fixed permutation which means the weight matrix is initialized to a random rotation matrix. A 1×1 invertible convolution with an equal number of input and output channels is a generalization of the permutation operation. The simplified calculation of the matrix simplifies the overall amount of calculation.

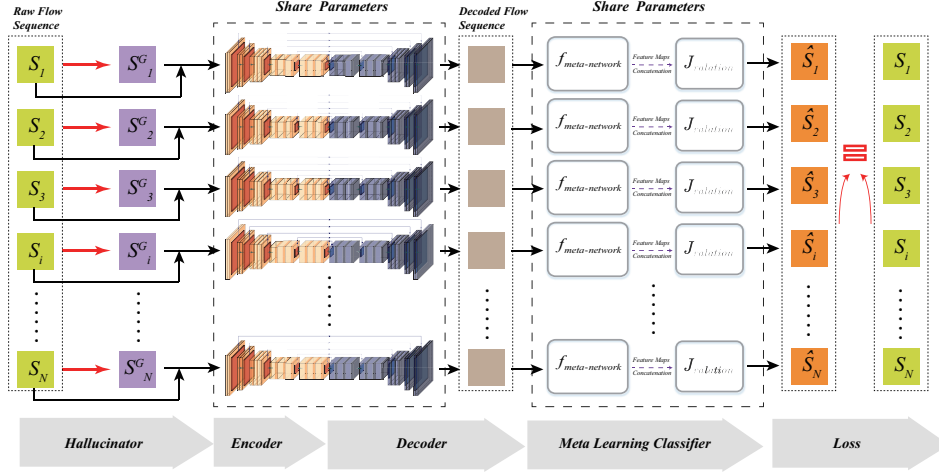


Figure 3: The whole architecture of Flow-Based Relation Network (RBRN) model. First, the Hallucinator generates a set of examples to create an augmented training flow set. Then, during the Encoder-Decoder Architecture, encoder generates compressed features, and decoder produces sparse feature map(s). All training samples share one encoder-decoder module. At last, Meta-Learning Based Classifier extracts features from decoded hallucinated flow sequences, and generates relation score and gives one-hot vector. Mean Square Error Loss is used for our model training.

2.3 AutoEncoder

AutoEncoder is a kind of Feedforward Neural Networks (NNs), which is mainly used for data dimensionality reduction or feature extraction, and is also extended to generative models. Firstly, unlike other Feedforward NNs focusing on the Output Layer and error rate, AutoEncoder focuses on the Hidden Layer. Secondly, the traditional Feedforward NN is generally deeper, and the AutoEncoder usually has only one layer of Hidden Layer.

The AutoEncoder structure consists of three parts, including Input Layer, Hidden Layer, and Output Layer. The constraints of this network are: The dimension of Hidden Layer is much smaller than the Input Layer; Output is used to reconstruct Input, which is to minimize the error L .

Thus, we can use the output of the Hidden Layer (called code) to represent input data, which achieves the effect of Input compression. The training method of AutoEncoder is same as traditional backpropagation. The part that compresses input data into code is called encoder, and the part that restores code to input data is called decoder. The total process is shown in Figure 2.

3 THE FLOW RELATION NETWORK

Our end-to-end Flow-Based Relation Network (RBRN) is a hierarchical model as shown in Figure 3. The RBRN considers both feature learning and classification together. In the following of this section, we will present each layer in detail.

3.1 Hallucinator

We now present our approach to few-shot learning-based classification by learning to hallucinate additional examples. We first describe how this hallucinator is used in testing, and then discuss how we train the hallucinator. The whole process of hallucinator is shown in Figure 4.

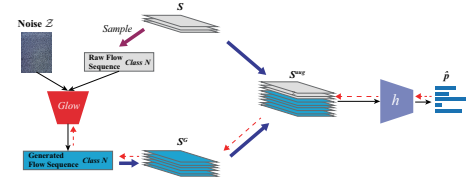


Figure 4: Meta-learning with hallucination. Given an initial training flow set S , we create an augmented training flow set S^{aug} by adding a set of generated examples S^G . S^G is obtained by sampling real seed flow examples and noise vectors Z , and passing them to a parametric hallucinator G . In addition, red arrows indicate the process of gradients during back-propagation.

Hallucination During Testing: During testing, we are given an initial training set S . We then hallucinate n_{gen} new examples using the hallucinator. Each hallucinated example is obtained by sampling a real example (x_i, y_i) from S , sampling a noise vector Z , and passing x_i and Z to Glow model to obtain a generated example (x'_i, y_i) where $x'_i = Glow(x_i, Z)$. We take the set of generated examples S^G and add it to the set of real examples to produce an augmented training set $S^{aug} = S \cup S^G$. We can now use this augmented training set to produce conditional probability estimates using h . Note that the hallucinator parameters are kept fixed here; any learning that happens, happens within the classification algorithm h .

Training the Hallucinator: The goal of the hallucinator is to produce examples that help the classification algorithm learn a better classifier. This goal differs from realism: realistic examples might still fail to capture the many modes of variation of visual concepts (categories), while unrealistic hallucinations can always

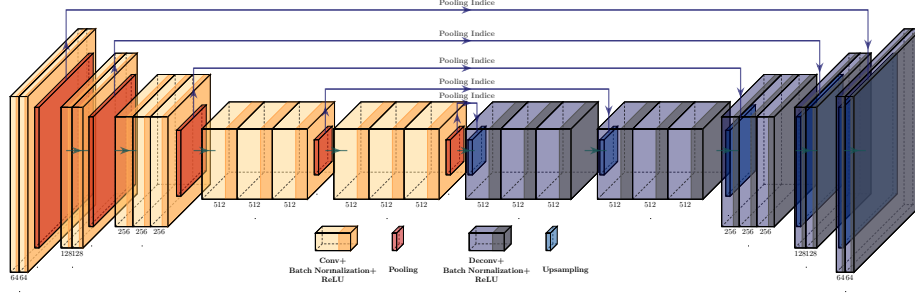


Figure 5: Architecture of encoder and decoder. The encoder takes the hallucination of flow as input, and generates the compressed features. The decoder produces sparse feature map(s). Encoder network consists of 13 convolutional layers. Each convolutional layer produces a set of feature maps, then followed by batch normalization and element-wise rectified-linear non-linearity (ReLU) $\max(0, x)$, at last applied max-pooling with a 2×2 window and stride 2 (non-overlapping window). The decoder is similar to the encoder and uses the memorized maxpooling indices from the corresponding encoder feature map(s).

lead to an ethical decision boundary [28]. We, therefore, propose to directly train the hallucinator to support the classification algorithm by using meta-learning. As before, in each meta-training iteration, we sample m classes from the set of all classes, and at most n examples per class. Then, for each class, we use *Glow* to generate n_{gen} additional examples until there are exactly n_{aug} examples per class. Again, each hallucinated example is of the form (x_i', y_i) , where $x_i' = \text{Glow}(x_i, \mathcal{Z})$, (x, y) is a sampled example from S and $\mathcal{Z}$ is a sampled noise vector. These additional examples are added to the training set S to produce an augmented training set S^{aug} . Then this augmented training set is fed to the classification algorithm h .

To train the hallucinator *Glow*, we require that the classification algorithm h is differentiable concerning the elements in S^{aug} . This is true for many meta-learning algorithms. For example, in our networks, h will pass every example in training set through a feature extractor, compute the class means in this feature space, and use the distances between the test point, and the class means to estimate class probabilities. If the feature extractor is differentiable, then the classification algorithm itself is differentiable concerning the examples in the training set.

Using meta-learning to train the hallucinator and the classification algorithm has two benefits. First, the hallucinator is directly trained to produce the kinds of hallucinations that are useful for class distinctions, removing the need to precisely tune realism or diversity, or the right modes of variation to hallucinate. Second, the classification algorithm is trained jointly with the hallucinator, which enables it to make allowances for any errors in the hallucination. Conversely, the hallucinator can spend its capacity on suppressing precisely those errors which throw the classification algorithm off.

3.2 Encoder and Decoder

The encoder takes the hallucination of flow as input, and generates the compressed features. The decoder is similar to the encoder. This architecture is illustrated in Figure 5. The encoder network consists of 13 convolutional layers which correspond to the first 13 convolutional layers in the VGG16 network [37] designed for our traffic classification. We can, therefore, initialize the training

process from weights trained for classification. We can also discard the fully connected layers in favor of retaining higher resolution feature maps at the deepest encoder output. Each encoder layer has a corresponding decoder layer, and hence the decoder network has 13 layers. The final decoder output is fed to our meta-learning-based classifier to produce class probabilities for encrypted traffic independently.

Encoder Architecture: Each *encoder* in the encoder network performs convolution with a filter bank to produce a set of feature maps. These are then batch normalized [30]. Then an element-wise rectified-linear non-linearity (ReLU) $\max(0, x)$ is applied. Following that, max-pooling with a 2×2 window and stride 2 (non-overlapping window) is performed, and the resulting output is sub-sampled by a factor of 2. Max pooling is used to achieve translation invariance over small spatial shifts in the input flow sequences. Sub-sampling results in a large input flow sequences' context (spatial window) in the feature map. While several layers of max-pooling and sub-sampling can achieve more translation invariance for robust classification correspondingly there is a loss of spatial resolution of the feature maps. Therefore, it is necessary to capture and store flow sequences' feature information in the encoder feature maps before sub-sampling is performed. If memory during inference is not constrained, then all the encoder feature maps (after sub-sampling) can be stored. This is usually not the case in practical applications, and hence, we propose a more efficient way to store this information. It involves storing only the max-pooling indices, i.e., the locations of the maximum feature value in each pooling window is memorized for each encoder feature map. In principle, this can be done using 2 bits for each 2×2 pooling window and is thus much more efficient to store as compared to memorizing feature map(s) in float precision.

Decoder Architecture: The appropriate *decoder* in the decoder network upsamples its input feature map(s) using the memorized maxpooling indices from the corresponding encoder feature map(s). This step produces sparse feature map(s). These feature maps are then convolved with a trainable decoder filter bank to produce dense feature maps. A batch normalization step is then applied to each of these maps. Note that the decoder corresponding to

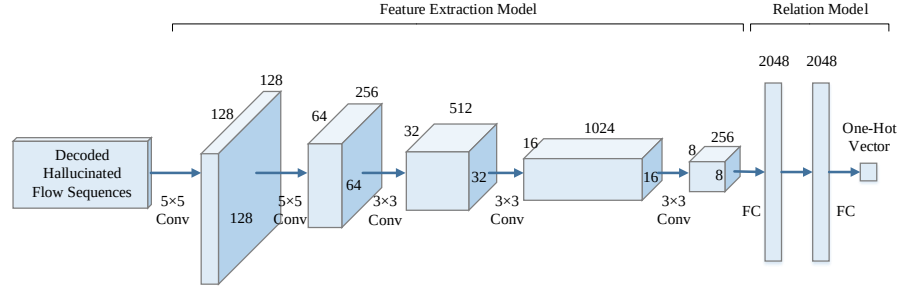


Figure 7: The classifier uses the 6-layer network architecture as feature extraction model. The output of the 6-th pooling layer can be regarded as network features. Then we apply the relation model.

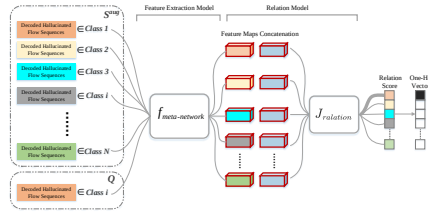


Figure 6: Meta-learning based classifier. It contains two modules: a feature extraction model and a relation model. The feature extraction model $f_{meta-network}$ produces feature maps to represent feature extraction function. The relation model $J_{relation}(\cdot)$ represents the similarity between sample and query.

the first encoder (closest to the input flow sequences) produces a multi-channel feature map. This is unlike the other decoders in the network which produce feature maps with the same number of size and channels as their encoder inputs. The high dimensional feature representation at the output of the final decoder is fed to our meta-learning-based classifier.

3.3 Classification of Meta-Learner

Meta-Learning Based Classifier: As illustrated in Figure 6, our matching network consists of two branches: a feature extraction model and a relation model. Suppose sample x_j in the query set Q and sample x_i in the sample set S^{aug} , we define the function $f_{Encoder\&Decoder}$ which represents feature extraction function using encoder-decoder network to produce handcrafted feature maps $f_{Encoder\&Decoder}(x_j)$ and $f_{Encoder\&Decoder}(x_i)$. Besides, we define the function $f_{meta-network}$ which represents feature extraction function using network to produce feature maps $f_{meta-network}(x_j)$ and $f_{meta-network}(x_i)$. The feature maps are combined using the function $C_{network}$, and the handcrafted feature maps are combined using the function $C_{Encoder\&Decoder}$. In this work, we assume the $C_{meta-network}(\cdot, \cdot)$ to be concatenation of corresponding feature maps in depth.

The combined feature map of the sample and query is used as the relation model $J_{relation}(\cdot)$ to get a scalar in range of 0 to

1 representing the similarity between x_i and x_j , which is called relation score. Suppose we have one labeled sample for each of n unique classes, our model can generate n relation scores $Judge_{i,j}$ for the relation between one query input x_j and training sample set examples x_i :

$$Judge_{i,j} = J_{relation}(C_{meta-network}(f_{meta-network}(f_{Encoder\&Decoder}(x_i)), f_{meta-network}(f_{Encoder\&Decoder}(x_j))))$$

$$i = 1, 2, \dots, n$$

(1)

Furthermore, for m labeled samples for each of n unique classes, we can element-wise sum over our feature extraction model outputs of all samples from each training class to form this class's feature map. And this pooled class-level feature map is combined with the query flow sequences feature map as above.

Our Meta-Learning Classifier Architecture: Figure 7 describes a traditional process of convolution and pooling. We use the 6-layer network architecture. Taking a decoded hallucinated flow sequences after encoding-decoding process as input, the output of the 6-th pooling layer is a 2048-dimensional vector, which we regard as network features. The kernels of network change in turns: $3 \times 256 \times 256 \rightarrow 128 \times 128 \times 128$ (Convolution, kernel size: 1×1) $\rightarrow 256 \times 64 \times 64$ (Convolution, kernel size: 3×3) $\rightarrow 512 \times 32 \times 32$ (Convolution, kernel size: 3×3) $\rightarrow 1024 \times 16 \times 16$ (Convolution, kernel size: 3×3) $\rightarrow 256 \times 8 \times 8$. Then, we apply the fully connected layer to change into 2048-dimensional vector.

3.4 Loss

We use mean square error (MSE) loss to train our model, regressing the relation score $Judge_{i,j}$ to the ground truth: matched pairs have similarity 1 and the mismatched pair have similarity 0.

$$Loss = \arg \min \sum_{i=1}^n \sum_{j=1}^m (Judge_{i,j} - (y_i == y_j))^2$$

(2)

4 EXPERIMENT AND RESULTS

In this section, we present the experimental results to evaluate the efficiency of the proposed RBRN. All experiments were conducted using a 4-core PC with an NVIDIA GTX 970 GPU, 16GB of RAM, and Ubuntu 16.

4.1 Experiment Settings

4.1.1 Dataset. We have not yet found an available public dataset that has both encrypted traffic and malware traffic, hence we decide to evaluate models using two public datasets ISCX 2012 IDS dataset [33] and ISCX VPN-nonVPN traffic dataset [8], respectively.

The first selected dataset is regenerated from ISCX VPN-nonVPN traffic dataset [8] in order to evaluate the effectiveness of encrypted traffic classification. The total dataset for evaluation is composed of 15 applications, e.g., Facebook, Youtube, Netflix, etc. The chosen applications are encrypted with various security protocols, including HTTPS, SSL, SSH, and proprietary protocols. A total of 206,688 data packets are included in the selected dataset. To reduce impacts from imbalanced dataset [8], e.g., Netflix accounts for 25.126% of the total dataset, while ICQ only accounts for 2.053%. We further create a subset with more balanced data samples for each application. The balanced subset has a total of 73,392 data packets. It is composed of the same 15 applications, where each class accounts for around 6.18% of the total subset.

The second dataset is regenerated from ISCX 2012 IDS dataset [8]. This dataset contains the network traffic of seven days, which can be divided into 5 classes, Normal, Brute Force SSH, DDoS, HttpDoS, and Infiltrating respectively. We select the data that only from days that have malware traffic for simplification. Since malware traffic is expected to have a relatively smaller scale compared to normal traffic in real life, no normalization approach on the dataset is applied. Finally, this selected dataset is divided into the training dataset and the testing dataset according to 9 : 1 for each class.

4.1.2 Evaluation Metrics. We evaluate and compare the performance of the our model with state-of-art methods using four metrics. Namely, Accuracy (Acc), Precision (Pr), Recall (Re), and F-Measure:

- Accuracy (Acc):

$$Acc = \frac{TP + TN}{TP + FP + FN + TN} \quad (3)$$

- Recall (Re) :

$$Re = \frac{TP}{TP + FN} \quad (4)$$

- Precision (Pr) :

$$Pr = \frac{TP}{TP + FP} \quad (5)$$

- F-Measure:

$$F - measure = \frac{2 \cdot Pr \cdot Re}{Pr + Re} \quad (6)$$

where TP is True Positive, namely the number of correctly classified cases as a specific class; FP is False Positive, namely the number of misclassified cases that classified as that class; FN, False Negative, which is the number of cases that are supposed to be classified as that class, yet misclassified as other classes; TN, True Negative, which is the number of cases that correctly classified as not that specific class.

4.1.3 Experimental Settings and Baselines. Settings We use the Adam optimizer with a batch size of 1, for training where the learning rate was set to 0.00001 and momentums were set to 0.5 and 0.999.

Baselines We compare against various state-of-the-art baselines for encrypted traffic classification, including METC [1], HEDGE [3], Laffit [9], HST [15], ACGAN [24], Datanet [27], ACNN [31], DeepFullRange [35] and LSTM [38].

4.2 Comparison Experiments

We use the *Balanced dataset* of ISCX VPN-nonVPN traffic dataset to design our comparison experiments. Table 1 is our results. Table 2 is the comparison results.

From Table 1 and 2, we can obtain the following conclusions:

(1) *RBRN achieves the best performance, and outperforms all the other methods. Besides, our RBRN can obtain the best performance on all the overall metrics, because it enjoys the advantages of the end-to-end learning architecture (i.e., joint learning of the feature representation and classification) and the Hallucinator and AutoEncoder/AutoDecoder mechanism.*

(2) *Our network's classification of traffic for different applications can maintain a high (more than 96%) accuracy. This shows that our network is highly robust.*

From these two points, we know that our algorithm has good robustness and accuracy.

4.3 Ablation Study

In order to verify the advancement and rationality of our model, we experimented with each component. We use the accuracy rate, precision, recall rate, and F-measure to evaluate the experimental results in the experiment on *Balanced* ISCX VPN-nonVPN traffic dataset. *From Table 3, we know our method is better than others. This shows that the design of our algorithm is reasonable.*

4.4 Discussion on Imbalanced Class Dataset

We use the *Full* ISCX VPN-nonVPN traffic dataset and ISCX 2012 IDS dataset, two imbalanced dataset, to design our experiments. Table 4 is our results on ISCX 2012 IDS dataset. Table 5 is the comparison results on ISCX 2012 IDS dataset. Table 6 is our results on full ISCX VPN-nonVPN traffic dataset. Table 7 is the comparison results on full ISCX VPN-nonVPN traffic dataset.

On ISCX 2012 IDS Dataset From Table 4 and 5, our approach maintains over 92% accuracy in traffic classification for each protocol. *More importantly, by comparing with other methods, we find that our method is best.*

On Full ISCX VPN-nonVPN Traffic Dataset From Table 6 and 7, our approach is effective for traffic classification of applications. By contrast, we find that our method is best. *It is worth mentioning that by comparing with the experimental data mentioned in Section 4.2, we find that whether it is balanced data or unbalanced data, we can get the best results on the application traffic classification problem.*

From these two points, we know that our algorithm has good robustness and accuracy, whether it is the traffic classification of the communication protocol or the traffic classification of the application.

4.5 Verification Experiment of Generalization Performance of Proposed Algorithm

To verify the generalization performance of our proposed algorithm when applied to other complex environments, we fix the model

Table 3: Results of Ablation Study on Balanced ISCX VPN-nonVPN traffic dataset

Method	RBRN w/o H&E-D				RBRN w/o H				RBRN w/o E-D				RBRN			
Hallucinator	$\times$				$\times$				$\sqrt$				$\sqrt$			
AutoEncoder/AutoDecoder	$\times$				$\sqrt$				$\times$				$\sqrt$			
	Acc	Pr	Re	F-Measure	Acc	Pr	Re	F-Measure	Acc	Pr	Re	F-Measure	Acc	Pr	Re	F-Measure
AIM	0.8131	0.8092	0.8130	0.8111	0.8216	0.8279	0.8441	0.8359	0.9264	0.9342	0.9255	0.9298	0.9701	0.9632	0.9698	0.9665
Email-Client	0.7934	0.8077	0.8115	0.8096	0.8255	0.8285	0.8246	0.8265	0.9241	0.9213	0.9374	0.9293	0.9663	0.9640	0.9752	0.9696
Facebook	0.8083	0.7952	0.8060	0.8005	0.8444	0.8325	0.8320	0.8323	0.9328	0.9373	0.9276	0.9324	0.9768	0.9652	0.9817	0.9734
Gmail	0.8101	0.8119	0.8108	0.8114	0.8238	0.8266	0.8337	0.8301	0.9207	0.9249	0.9420	0.9334	0.9637	0.9667	0.9713	0.9690
Hangout	0.7959	0.8128	0.8029	0.8079	0.8210	0.8243	0.8355	0.8299	0.9374	0.9206	0.9348	0.9276	0.9750	0.9832	0.9790	0.9811
ICQ	0.8100	0.7958	0.7967	0.7963	0.8309	0.8261	0.8213	0.8237	0.9436	0.9246	0.9444	0.9344	0.9731	0.9832	0.9777	0.9804
Netflix	0.8082	0.8100	0.8076	0.8088	0.8438	0.8287	0.8202	0.8244	0.9282	0.9266	0.9245	0.9256	0.9833	0.9672	0.9773	0.9722
SCP	0.7923	0.8028	0.7985	0.8006	0.8305	0.8288	0.8227	0.8257	0.9383	0.9390	0.9257	0.9323	0.9738	0.9737	0.9807	0.9772
SFTP	0.7928	0.7959	0.8068	0.8013	0.8375	0.8218	0.8370	0.8293	0.9217	0.9326	0.9232	0.9279	0.9744	0.9678	0.9695	0.9687
Skype	0.7956	0.8082	0.8069	0.8075	0.8249	0.8264	0.8239	0.8252	0.9337	0.9344	0.9396	0.9370	0.9739	0.9656	0.9727	0.9692
Spotify	0.8009	0.8069	0.8088	0.8079	0.8248	0.8214	0.8265	0.8240	0.9355	0.9352	0.9292	0.9322	0.9748	0.9838	0.9743	0.9791
torTwitter	0.7915	0.8148	0.8050	0.8099	0.8220	0.8436	0.8244	0.8339	0.9306	0.9262	0.9248	0.9255	0.9600	0.9725	0.9686	0.9705
Vimeo	0.8067	0.8014	0.7944	0.7979	0.8357	0.8304	0.8203	0.8253	0.9350	0.9415	0.9429	0.9422	0.9637	0.9615	0.9731	0.9672
voipbuster	0.7907	0.7933	0.7973	0.7953	0.8275	0.8213	0.8360	0.8286	0.9391	0.9277	0.9277	0.9277	0.9721	0.9628	0.9646	0.9637
Youtube	0.7998	0.8008	0.7915	0.7961	0.8234	0.8259	0.8263	0.8261	0.9243	0.9427	0.9214	0.9319	0.9688	0.9828	0.9640	0.9733
Total	0.8006	0.8044	0.8039	0.8041	0.8292	0.8276	0.8286	0.8281	0.9314	0.9313	0.9314	0.9313	0.9713	0.9709	0.9733	0.9721

Table 1: The Results of Balanced ISCX VPN-nonVPN traffic dataset

	Acc	Pr	Re	F-Measure
AIM	0.9701	0.9632	0.9698	0.9665
Email-Client	0.9663	0.9640	0.9752	0.9696
Facebook	0.9768	0.9652	0.9817	0.9734
Gmail	0.9637	0.9667	0.9713	0.9690
Hangout	0.9750	0.9832	0.9790	0.9811
ICQ	0.9731	0.9832	0.9777	0.9804
Netflix	0.9833	0.9672	0.9773	0.9722
SCP	0.9738	0.9737	0.9807	0.9772
SFTP	0.9744	0.9678	0.9695	0.9687
Skype	0.9739	0.9656	0.9727	0.9692
Spotify	0.9748	0.9838	0.9743	0.9791
torTwitter	0.9600	0.9725	0.9686	0.9705
Vimeo	0.9637	0.9615	0.9731	0.9672
voipbuster	0.9721	0.9628	0.9646	0.9637
Youtube	0.9688	0.9828	0.9640	0.9733
Total	0.9713	0.9709	0.9733	0.9721

Table 2: Comparison Results on Balanced ISCX VPN-nonVPN traffic dataset

	Acc	Pr	Re	F-Measure
RBRN	0.9713	0.9709	0.9733	0.9721
METC[1]	0.9703	0.9486	0.8599	0.9021
HEDGE[3]	0.8308	0.9366	0.8488	0.8905
Laffit[9]	0.9216	0.9692	0.8739	0.9191
HST[15]	0.8723	0.9158	0.8349	0.8735
ACGAN[24]	0.9267	0.9586	0.9078	0.9325
Datanet[27]	0.9103	0.9630	0.8628	0.9101
ACNN[31]	0.8528	0.8747	0.8342	0.8540
DeepFullRange[35]	0.8280	0.8577	0.9158	0.8858
LSTM[38]	0.8820	0.9437	0.8670	0.9038

Table 4: The Results of RBRN on ISCX 2012 IDS dataset

	Acc	Pr	Re	F-Measure
Normal	0.9512	0.9541	0.9617	0.9579
Brute Force SSH	0.9581	0.9630	0.9655	0.9643
DDoS	0.9641	0.9516	0.9605	0.9560
HttpDoS	0.9547	0.9595	0.9505	0.9550
Infiltrating Transfer	0.9535	0.9561	0.9568	0.9565
Total	0.9563	0.9569	0.9590	0.9579

Table 5: Comparison Results on ISCX 2012 IDS dataset

	Acc	Pr	Re	F-Measure
RBRN	0.9563	0.9569	0.9590	0.9579
METC[1]	0.9462	0.8658	0.9585	0.9098
HEDGE[3]	0.9302	0.9310	0.8561	0.8920
Laffit[9]	0.8501	0.8502	0.9583	0.9010
HST[15]	0.8232	0.8483	0.8992	0.8730
ACGAN[24]	0.8560	0.8329	0.8352	0.8340
Datanet[27]	0.8547	0.9459	0.8508	0.8959
ACNN[31]	0.9263	0.9516	0.9167	0.9338
DeepFullRange[35]	0.9259	0.8990	0.8785	0.8886
LSTM[38]	0.8765	0.8102	0.8932	0.8497

trained on the *Balanced* ISCX VPN-nonVPN traffic dataset (i.e., without any extra training or finetuning) and test its performance on the *Full* ISCX VPN-nonVPN traffic dataset. Table 8 is the results of verification experiment.

From Table 8, by comparing with other methods, we argue that the generalization performance of our algorithm is higher than other methods. Once more, it is worthy to point out that our model used in this section is trained on the *Balanced* ISCX VPN-nonVPN traffic dataset, without getting any training or finetuning on the *Full* ISCX VPN-nonVPN traffic dataset. This gives a clear signal that even though

Table 6: The Results of RBRN on Full ISCX VPN-nonVPN traffic dataset

	Acc	Pr	Re	F-Measure
AIM	0.9452	0.9536	0.9500	0.9518
Email-Client	0.9412	0.9529	0.9428	0.9478
Facebook	0.9428	0.9394	0.9536	0.9464
Gmail	0.9405	0.9304	0.9331	0.9318
Hangout	0.9353	0.9412	0.9378	0.9395
ICQ	0.9530	0.9498	0.9434	0.9466
Netflix	0.9340	0.9477	0.9355	0.9416
SCP	0.9374	0.9344	0.9317	0.9330
SFTP	0.9499	0.9544	0.9459	0.9501
Skype	0.9492	0.9354	0.9501	0.9427
Spotify	0.9305	0.9329	0.9403	0.9366
torTwitter	0.9465	0.9390	0.9518	0.9453
Vimeo	0.9550	0.9475	0.9448	0.9461
voipbuster	0.9498	0.9519	0.9409	0.9464
Youtube	0.9377	0.9378	0.9303	0.9340
Total	0.9432	0.9432	0.9421	0.9427

Table 7: Comparison Results on Full ISCX VPN-nonVPN traffic dataset

	Acc	Pr	Re	F-Measure
RBRN	0.9432	0.9432	0.9421	0.9427
METC[1]	0.8757	0.9176	0.9329	0.9252
HEDGE[3]	0.8514	0.9324	0.8689	0.8995
Lafft[9]	0.9059	0.9258	0.8985	0.9120
HST[15]	0.9206	0.9000	0.8980	0.8990
ACGAN[24]	0.9127	0.8603	0.8753	0.8677
Datanet[27]	0.9212	0.8723	0.8130	0.8416
ACNN[31]	0.9145	0.8117	0.8993	0.8533
DeepFullRange[35]	0.8573	0.8559	0.9316	0.8921
LSTM[38]	0.8619	0.9139	0.8399	0.8753

our approach belongs to the deep learning category, it has an amazing ability to generalize to unseen data, and has good robustness.

Table 8: The Results of Verification Experiment

	Acc	Pr	Re	F-Measure
RBRN	0.9212	0.9212	0.9201	0.9207
METC[1]	0.7789	0.8261	0.8814	0.8529
HEDGE[3]	0.9100	0.7084	0.9027	0.7938
Lafft[9]	0.9008	0.7813	0.7272	0.7532
HST[15]	0.9054	0.8599	0.7798	0.8179
ACGAN[24]	0.6874	0.8155	0.8445	0.8297
Datanet[27]	0.7271	0.6767	0.9078	0.7754
ACNN[31]	0.8351	0.7679	0.7772	0.7726
DeepFullRange[35]	0.8260	0.8384	0.7481	0.7907
LSTM[38]	0.6718	0.7247	0.8436	0.7797

4.6 Discussion on Small Dataset

In order to verify the effectiveness and robustness of the proposed algorithm in small samples, we design the comparison experiment. In Full ISCX VPN-nonVPN traffic dataset, we choose 1000 traffic records for each application as a training set, and the others as test sets. Table 9 is the comparison results under small samples.

From Table 9, it is clear that our method once again achieves the best results among all methods. This suggests that our method is more effective than other methods for few-shot encrypted traffic classification.

Table 9: Comparison Results under Small Samples

	Acc	Pr	Re	F-Measure
RBRN	0.8772	0.8772	0.8761	0.8767
METC[1]	0.7005	0.6523	0.7444	0.6953
HEDGE[3]	0.6607	0.6594	0.7466	0.7003
Lafft[9]	0.7135	0.7886	0.7182	0.7517
HST[15]	0.7614	0.7919	0.7455	0.7680
ACGAN[24]	0.7735	0.8718	0.8574	0.8646
Datanet[27]	0.7206	0.7224	0.6612	0.6904
ACNN[31]	0.8432	0.8765	0.8253	0.8502
DeepFullRange[35]	0.8430	0.6966	0.8240	0.7549
LSTM[38]	0.6654	0.7255	0.7902	0.7565

5 CONCLUSION

In this paper, we design an end-to-end encrypted traffic classification model named RBRN. It jointly learns the representative features from the raw flow sequences and classifies these flows together. The RBRN takes hallucinator maps real flow sequences to hallucinated flow sequences, and then takes a multi-layer encoder to learn the representation of the hallucinated flow sequence, reconstructs the hallucinated sequence with a multi-layer decoder. The features learned from the encoder and decoder are combined for classification. The end-to-end framework makes the RBRN learn representative information via meta-learning. We validate the effectiveness of the RBRN on the real-world network traffic dataset, and the experimental results demonstrate that the RBRN can achieve an excellent classification performance and outperform the state-of-the-art methods on encrypted traffic classification. What is more interesting, our model trained on real-world dataset can generalize very well to unseen datasets, outperforming multiple state-of-art methods. Moreover, our model has an amazing ability to classify few-shot encrypted traffic. In the future, we would like to investigate the ability of classification algorithms to predict samples from new classes when they are trained on the synthesized data generated by RBRN. Moreover, we also want to examine and apply recent advance in deep learning to improve the results of the machine learning algorithm in network traffic analysis.

ACKNOWLEDGMENTS

This work is supported by National Natural Science Foundation of China (61533019, 61806198, U1811463).

REFERENCES

- [1] G. Aceto, D. Ciunzio, A. Montieri, and A. Pescapé. 2019. Mobile Encrypted Traffic Classification Using Deep Learning: Experimental Evaluation, Lessons Learned, and Challenges. *IEEE Transactions on Network and Service Management* 16, 2 (2019), 445–458. <https://doi.org/10.1109/Tnsn.2019.2899085>
- [2] Giuseppe Aceto, Domenico Ciunzio, Antonio Montieri, and Antonio Pescapé. 2018. Multi-classification approaches for classifying mobile app traffic. *Journal of Network and Computer Applications* 103 (2018), 131 – 145. <https://doi.org/10.1016/j.jnca.2017.11.007>
- [3] F. Casino, K. K. R. Choo, and C. Patsakis. 2019. HEDGE: Efficient Traffic Classification of Encrypted and Compressed Packets. *IEEE Transactions on Information Forensics And Security* 14, 11 (2019), 2916–2926. <https://doi.org/10.1109/Tifs.2019.2911156>
- [4] Klenilmar Lopes Dias, Mateus Almeida Pongelupé, Waldir Matos Caminhas, and Luciano de Errico. 2019. An innovative approach for real-time network traffic classification. *Computer Networks* 158 (2019), 143 – 157. <https://doi.org/10.1016/j.comnet.2019.04.004>
- [5] K. Flanagan, E. Fallon, P. Jacob, A. Awad, and P. Connolly. 2019. 2D2N: A Dynamic Degenerative Neural Network for Classification of Images of Live Network Data. In *2019 16th IEEE Annual Consumer Communications Networking Conference (CCNC)*. 1–7. <https://doi.org/10.1109/CCNC.2019.8651695>
- [6] Jonathan Ho, Xi Chen, Aravind Srinivas, Yan Duan, and Pieter Abbeel. 2019. Flow++: Improving Flow-Based Generative Models with Variational Dequantization and Architecture Design. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.), Vol. 97. PMLR, Long Beach, California, USA, 2722–2730. <http://proceedings.mlr.press/v97/ho19a.html>
- [7] Durk P Kingma and Prafulla Dhariwal. 2018. Glow: Generative Flow with Invertible 1x1 Convolutions. In *Advances in Neural Information Processing Systems 31*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.). Curran Associates, Inc., 10215–10224. <http://papers.nips.cc/paper/8224-glow-generative-flow-with-invertible-1x1-convolutions.pdf>
- [8] Arash Habibi Lashkari, Gerard Draper-Gil, Mohammad Saiful Islam Mamun, and Ali A Ghorbani. 2017. Characterization of Tor Traffic using Time based Features. In *ICISSP*. 253–262.
- [9] Chang Liu, Zigang Cao, Zhen Li, and Gang Xiong. [n.d.]. Laffit: Length-aware fft based fingerprinting for encrypted network traffic classification. In *2018 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 1–6.
- [10] C. Liu, Z. Cao, G. Xiong, G. Gou, S. Yiu, and L. He. 2018. MaMPF: Encrypted Traffic Classification Based on Multi-Attribute Markov Probability Fingerprints. In *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*. 1–10. <https://doi.org/10.1109/IWQoS.2018.8624124>
- [11] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li. 2019. FS-Net: A Flow Sequence Network For Encrypted Traffic Classification. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*. 1171–1179. <https://doi.org/10.1109/INFOCOM.2019.8737507>
- [12] Zhen Liu, Ruoyu Wang, Nathalie Japkowicz, Yongming Cai, Deyu Tang, and Xianfa Cai. 2019. Mobile app traffic flow feature extraction and selection for improving classification robustness. *Journal of Network and Computer Applications* 125 (2019), 190 – 208. <https://doi.org/10.1016/j.jnca.2018.10.018>
- [13] T. Mangla, E. Halepovic, M. Ammar, and E. Zegura. 2019. Using Session Modeling to Estimate HTTP-based Video QoE Metrics from Encrypted Network Traffic. *IEEE Transactions on Network and Service Management* (2019), 1–1. <https://doi.org/10.1109/TNSM.2019.2924942>
- [14] A. Montieri, D. Ciunzio, G. Bovenzi, V. Persico, and A. Pescapé. 2019. A Dive into the Dark Web: Hierarchical Traffic Classification of Anonymity Tools. *IEEE Transactions on Network Science and Engineering* (2019), 1–1. <https://doi.org/10.1109/TNSE.2019.2901994>
- [15] W. N. Niu, Z. L. Zhuo, X. S. Zhang, X. J. Du, G. W. Yang, and M. Guizani. 2019. A Heuristic Statistical Testing Based Approach for Encrypted Network Traffic Identification. *IEEE Transactions on Vehicular Technology* 68, 4 (2019), 3843–3853. <https://doi.org/10.1109/Tvt.2019.2894290>
- [16] Antônio J. Pinheiro, Jeandro de M. Bezerra, Caio A.P. Burgardt, and Divanilson R. Campelo. 2019. Identifying IoT devices and events based on packet length from encrypted traffic. *Computer Communications* 144 (2019), 8 – 17. <https://doi.org/10.1016/j.comcom.2019.05.012>
- [17] R. Prenger, R. Valle, and B. Catanzaro. 2019. Waveglow: A Flow-based Generative Network for Speech Synthesis. In *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 3617–3621. <https://doi.org/10.1109/ICASSP.2019.8683143>
- [18] S. Rezaei and X. Liu. 2019. Deep Learning for Encrypted Traffic Classification: An Overview. *IEEE Communications Magazine* 57, 5 (May 2019), 76–81. <https://doi.org/10.1109/MCOM.2019.1800819>
- [19] Mert Bulent Sariyildiz and Ramazan Gokberk Cinbis. 2019. Gradient Matching Generative Networks for Zero-Shot Learning. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [20] Satadal Sengupta, Niloy Ganguly, Pradipta De, and Sandip Chakraborty. 2019. Exploiting Diversity in Android TLS Implementations for Mobile App Traffic Classification. In *The World Wide Web Conference (WWW '19)*. ACM, New York, NY, USA, 1657–1668. <https://doi.org/10.1145/3308558.3313738>
- [21] Guanglu Sun, Teng Chen, Yangyang Su, and Chenglong Li. 2018. Internet Traffic Classification Based on Incremental Support Vector Machines. *Mobile Networks and Applications* 23, 4 (01 Aug 2018), 789–796. <https://doi.org/10.1007/s11036-018-0999-x>
- [22] Flood Sung, Yongxin Yang, Li Zhang, Tao Xiang, Philip H.S. Torr, and Timothy M. Hospedales. 2018. Learning to Compare: Relation Network for Few-Shot Learning. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [23] R. Vrána, J. Kořenek, and D. Novák. 2019. Acceleration of Feature Extraction for Real-Time Analysis of Encrypted Network Traffic. In *2019 IEEE 22nd International Symposium on Design and Diagnostics of Electronic Circuits Systems (DDECS)*. 1–6. <https://doi.org/10.1109/DDECS.2019.8724658>
- [24] Ly Vu, Cong Thanh Bui, and Quang Uy Nguyen. [n.d.]. A deep learning based method for handling imbalanced problem in network traffic classification. In *Proceedings of the Eighth International Symposium on Information and Communication Technology*. ACM, 333–339.
- [25] L. Vu, H. V. Thuy, Q. U. Nguyen, T. N. Ngoc, D. N. Nguyen, D. T. Hoang, and E. Dutkiewicz. 2018. Time Series Analysis for Encrypted Traffic Classification: A Deep Learning Approach. In *2018 18th International Symposium on Communications and Information Technologies (ISCIT)*. 121–126. <https://doi.org/10.1109/ISCIT.2018.8587975>
- [26] P. Wang, X. Chen, F. Ye, and Z. Sun. 2019. A Survey of Techniques for Mobile Service Encrypted Traffic Classification Using Deep Learning. *IEEE Access* 7 (2019), 54024–54033. <https://doi.org/10.1109/ACCESS.2019.2912896>
- [27] P. Wang, F. Ye, X. J. Chen, and Y. Qian. 2018. Datanet: Deep learning Based Encrypted Network Traffic Classification in SDN Home Gateway. *IEEE Access* 6 (2018), 55380–55391. <https://doi.org/10.1109/Access.2018.2872430>
- [28] Yu-Xiong Wang, Ross Girshick, Martial Hebert, and Bharath Hariharan. 2018. Low-Shot Learning From Imaginary Data. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [29] Xi Xiao, Rui Li, Hai-Tao Zheng, Runguo Ye, Arun KumarSangaiah, and Shutao Xia. 2019. Novel dynamic multiple classification system for network traffic. *Information Sciences* 479 (2019), 526 – 541. <https://doi.org/10.1016/j.ins.2018.10.039>
- [30] X. Xiao, L. Wang, K. Ding, S. Xiang, and C. Pan. 2019. Deep Hierarchical Encoder-Decoder Network for Image Captioning. *IEEE Transactions on Multimedia* (2019), 1–1. <https://doi.org/10.1109/TMM.2019.2915033>
- [31] Ying Yang, Cuicui Kang, Gaopeng Gou, Zhen Li, and Gang Xiong. [n.d.]. TLS/SSL Encrypted Traffic Classification with Autoencoder and Convolutional Neural Network. In *2018 IEEE 20th International Conference on High Performance Computing and Communications; IEEE 16th International Conference on Smart City; IEEE 4th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*. IEEE, 362–369.
- [32] H. Yao, P. Gao, J. Wang, P. Zhang, C. Jiang, and Z. Han. 2019. Capsule Network Assisted IoT Traffic Classification Mechanism for Smart Cities. *IEEE Internet of Things Journal* (2019), 1–1. <https://doi.org/10.1109/IJOT.2019.2901348>
- [33] Xinyou Yin, JAN Goudriaan, Egbert A Lantinga, JAN Vos, and Huub J Spiertz. 2003. A flexible sigmoid function of determinate growth. *Annals of botany* 91, 3 (2003), 361–371.
- [34] X. Zeng, X. Chen, G. Shao, T. He, Z. Han, Y. Wen, and Q. Wang. 2019. Flow Context and Host Behavior Based Shadowsocks's Traffic Identification. *IEEE Access* 7 (2019), 41017–41032. <https://doi.org/10.1109/ACCESS.2019.2907149>
- [35] Y. Zeng, H. X. Gu, W. T. Wei, and Y. T. Guo. 2019. Deep-Full-Range: A Deep Learning Based Network Encrypted Traffic Classification and Intrusion Detection Framework. *IEEE Access* 7 (2019), 45182–45190. <https://doi.org/10.1109/Access.2019.2908225>
- [36] J. Zheng and D. Li. 2019. GCN-TC: Combining Trace Graph with Statistical Features for Network Traffic Classification. In *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*. 1–6. <https://doi.org/10.1109/ICC.2019.8761115>
- [37] Alex Zihao Zhu, Liangzhe Yuan, Kenneth Chaney, and Kostas Daniilidis. 2019. Unsupervised Event-Based Learning of Optical Flow, Depth, and Egomotion. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [38] Zhuang Zou, Jingguo Ge, Hongbo Zheng, Yulei Wu, Chunjing Han, and Zhongjiang Yao. [n.d.]. Encrypted Traffic Classification with a Convolutional Long Short-Term Memory Neural Network. In *2018 IEEE 20th International Conference on High Performance Computing and Communications; IEEE 16th International Conference on Smart City; IEEE 4th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*. IEEE, 329–334.